

Розділ 1. Створення діаграм компонентів, взаємодії та класів веб-додатку

1.1. Опис функціональних можливостей (Діаграма варіантів використання)

На основі розробленої діаграми реалізовано наступні можливості:

1. Користувач (та Гість) має можливість генерувати надійні паролі, для цього буде реалізовано:

- а. Сторінку генератора паролів з налаштуваннями довжини та набору символів.
- б. Можливість перевірки надійності пароля (ентропії) в реальному часі.

2. Користувач має можливість увійти в обліковий запис, попередньо його створивши, для цього буде реалізовано:

- а. Сторінку реєстрації з валідацією електронної адреси (має бути унікальною) та майстер-пароля (мін. 12 символів, наявність цифр та спецсимволів).
- б. Сторінку авторизації з переадресацією на головну сторінку після успішного входу.
- с. Систему відображення помилок: про невірні дані або про вже існуючу в системі адресу.

3. Зареєстрований користувач має можливість керувати своїми даними, для цього буде реалізовано:

- а. Сторінку особистого кабінету зі списком збережених сервісів.
- б. Форму додавання нового запису (назва сервісу, логін, пароль).
- с. Можливість перегляду історії попередньо створених записів.

4. Адміністратор має можливість керувати системою, для цього буде реалізовано:

- а. Панель адміністратора для перегляду списку зареєстрованих користувачів.
- б. Можливість блокування або видалення облікових записів.
- с. Доступ до загальної статистики використання системи (без доступу до приватних паролів).

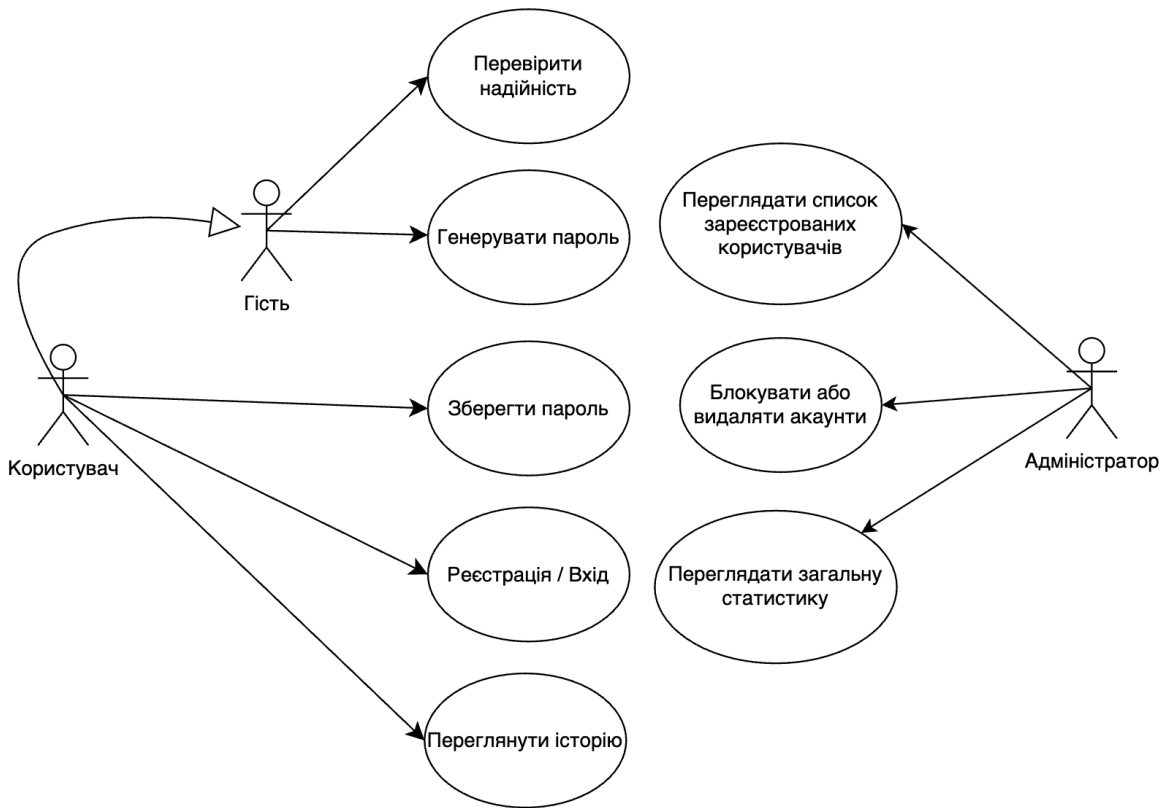


Рисунок 1. Діаграма варіантів використання

1.2. Опис процесу збереження даних (Діаграма взаємодії)

Для забезпечення безпеки під час збереження пароля реалізовано наступний алгоритм:

1. Користувач ініціює збереження через браузер.
2. Дані передаються на сервер Django через POST-запит.
3. Сервер виконує шифрування даних за алгоритмом AES-256 перед записом у базу.
4. Зашифровані дані (`encrypted_data`) записуються в PostgreSQL.
5. Користувач отримує підтвердження про успішне збереження.

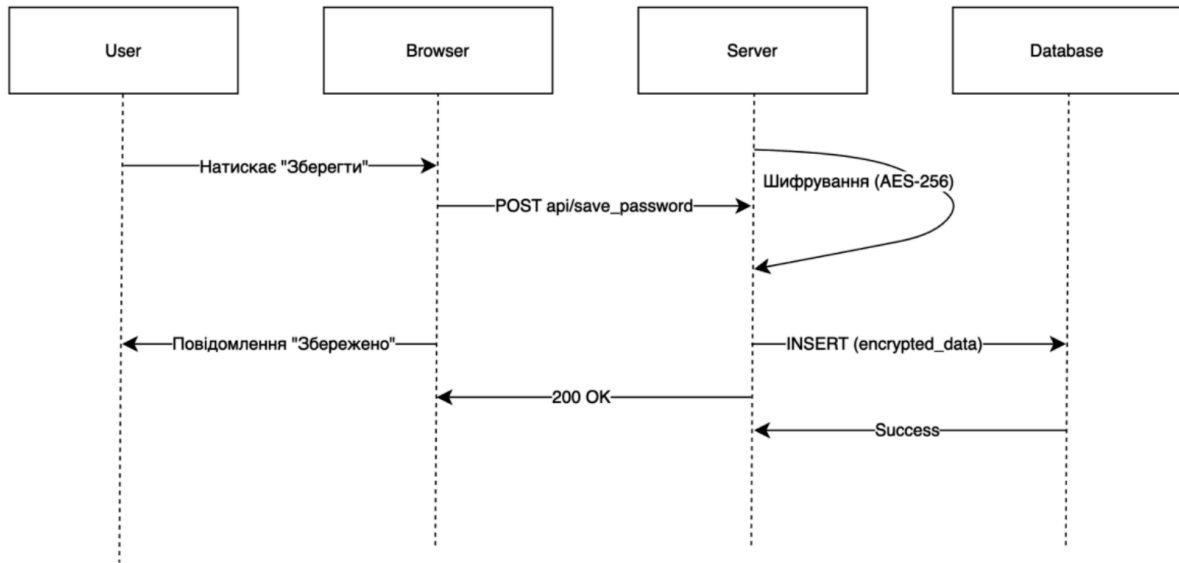


Рисунок 2. Діаграма взаємодії

1.3. Технічна архітектура та структура коду (Діаграми класів та компонентів)

Проектування класів: Логіка додатка базується на взаємодії класів User, PasswordRecord та сервісів шифрування CryptoManager. Клас GeneratorService надає інструментарій для створення паролів, а CryptoManager забезпечує безпеку через методи encrypt_aes та decrypt_aes.

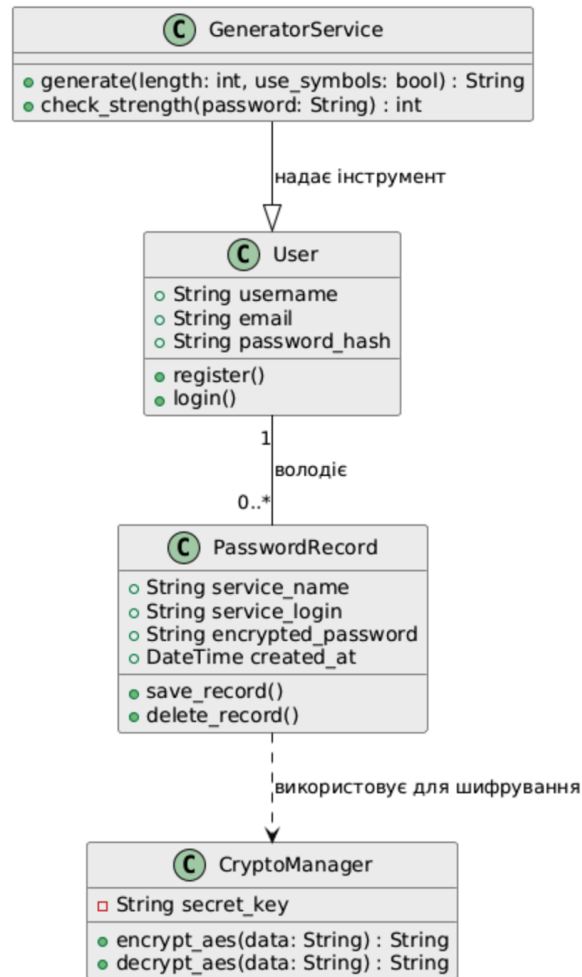


Рисунок 3. Діаграми класів

Розподіл на компоненти: Система розділена на клієнтську частину (Web Interface), серверну логіку (Django) та базу даних (PostgreSQL). Серверна частина включає окремі модулі авторизації (Auth Module) та менеджер паролів, який використовує Encryption Engine для захисту даних.

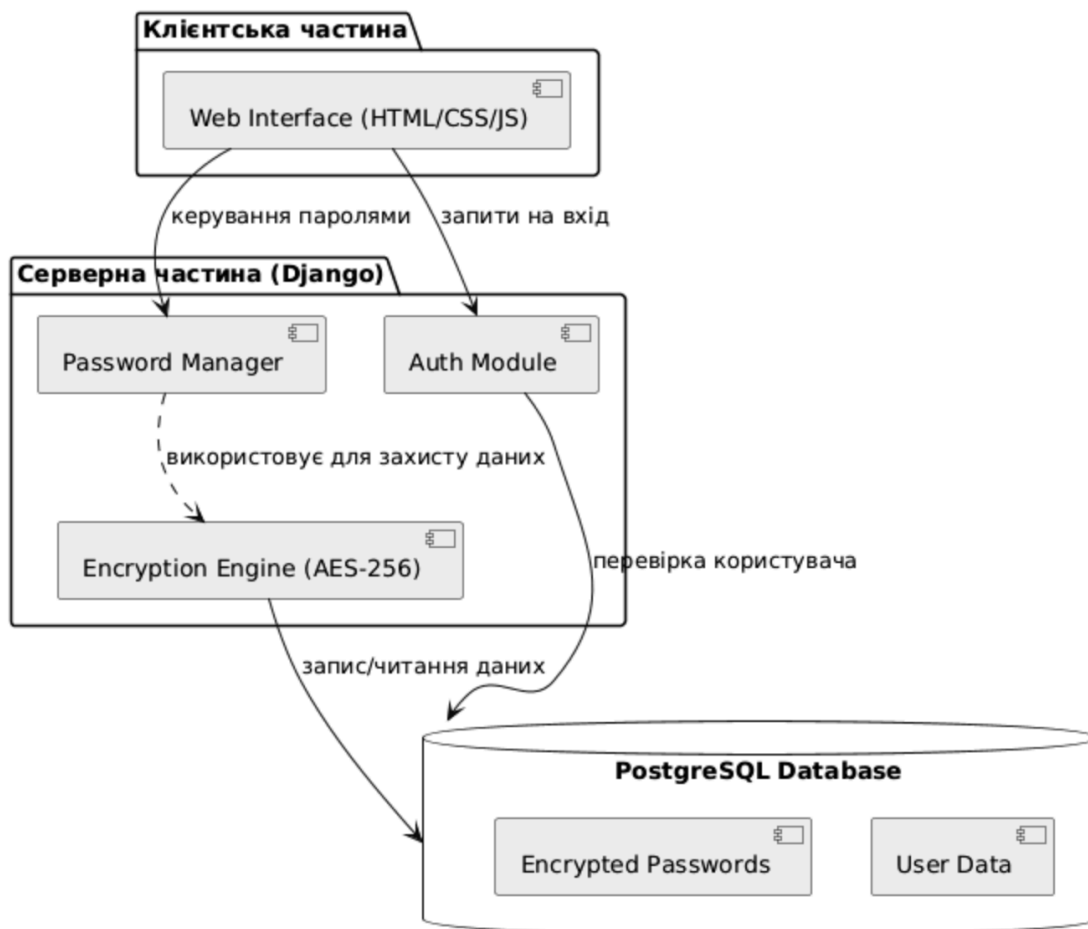


Рисунок 4. Діаграми компонентів

Розділ 2. Проектування структури меню з урахуванням потреб користувача.

Меню додатку для створення безпечних паролів буде включати наступні пункти:

- Генератор (головна)
- Збережені паролі
- Перевірка надійності
- Особистий кабінет
- Про проект

В комп'ютерній версії веб-додатку меню буде знаходитись в лівій частині екрану для зручного доступу. Також передбачена верхня панель, де розташовані логотип та кнопка виходу з облікового запису.

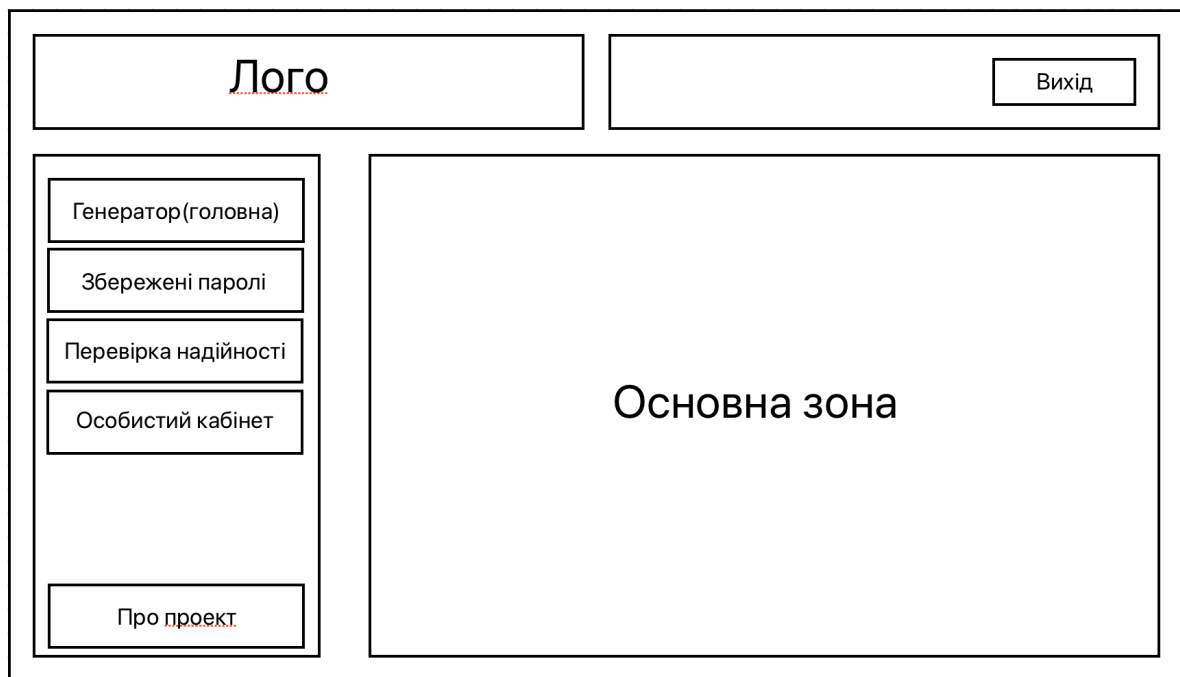


Рисунок 5. Схематичне зображення розміщення меню у веб-додатку.

В мобільній версії меню буде знаходитись між розділом з логотипом та основним блоком.



Рисунок 6. Розміщення елементів меню у мобільній версії веб-додатку.

1. Генератор (головна)

а. Елемент меню буде переадресовувати користувача на головну сторінку, де буде відображатись інтерфейс для генерації нових безпечних паролів.

2. Збережені паролі

а. Елемент меню буде переадресовувати користувача на сторінку перегляду зашифрованих паролів від сервісів, які були збережені цим користувачем.

3. Перевірка надійності

а. Елемент меню буде переадресовувати користувача на сторінку з інструментом для перевірки ентропії (складності) існуючих паролів.

4. Особистий кабінет

а. Елемент меню буде переадресовувати користувача на сторінку з інформацією про обліковий запис користувача та його налаштуваннями.

5. Про проект

а. Елемент меню буде переадресовувати користувача на сторінку з інформацією про даний веб-додаток та використані алгоритми захисту даних.

Розділ 3. Розміщення та стилізація елементів інтерфейсу.

1. Сторінка створення облікового запису користувача

а. На сторінці присутні чотири поля для введення даних:

- Ім'я.
- Електронна адреса
- Пароль
- Повторне введення паролю

Та дві кнопки

- Створення облікового запису
- Та переадресація на сторінку авторизації

The diagram illustrates the layout of a user registration form. At the top center is a rounded rectangular box labeled "Logo". Below it is a vertical container with a rectangular border. Inside this container, the fields are stacked vertically: a "Name" label followed by a text input field, an "Email" label followed by a text input field, a "Password" label followed by a text input field, and a "Repeat password" label followed by a text input field. At the bottom of this container are two buttons: "Have an account?" on the left and "Sign up" on the right.

Рисунок 7. Схема розміщення елементів інтерфейсу сторінки створення облікового запису користувача

2. Сторінка створення авторизації

а. На сторінці присутні два поля для ведення даних:

- Електронна адреса
- Пароль

Та три кнопки

- Забули пароль
- Вхід в систему
- Та переадресація на сторінку реєстрації

Рисунок 8. Схема розміщення елементів інтерфейсу сторінки авторизації

Розділ 4. Проектування та реалізація моделей даних для збереження інформації.

В системі передбачено наступні сутності даних:

- Користувач
- Категорія
- Збережений запис

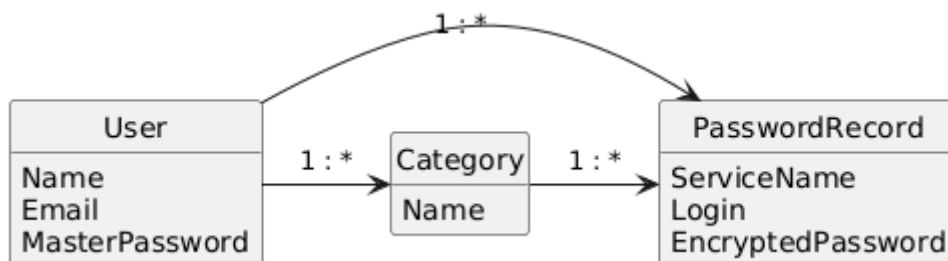


Рисунок 9. Діаграма моделей даних для збереження інформації

Модель даних “Користувач” зберігає в собі необхідні атрибути для користувачів системи. Користувач може створювати власні папки для сортування та безпосередньо зашифровані облікові дані, які відображаються моделями даних “Категорія” та “Збережений запис” відповідно.

Залежності між моделями:

- Один збережений запис може стосуватись тільки однієї категорії.
- Один збережений запис може мати тільки одного власника (автора).
- Один користувач може створювати багато категорій та збережених записів.
- Одна категорія може містити багато збережених записів.

Розділ 5. Розробка механізмів отримання та оновлення даних

У даному веб-додатку механізми отримання та оновлення даних реалізовані через класичні HTTP-запити (GET та POST) з використанням архітектури Django.

На основі розробленої діаграми реалізовано наступні можливості:

1. Авторизація та Реєстрація. Після того, як користувач заповнює форму входу або реєстрації, відбувається POST-запит на сервер за шляхом /login/ або /register/, куди передаються введені дані. У випадку успішної валідації (статус 200) виконується переадресація користувача на головну сторінку генератора або в особистий кабінет. Якщо дані введені некоректно (або така пошта вже існує), сервер повертає сторінку з відповідним повідомленням про помилку, щоб користувач міг виправити дані.

2. Збереження згенерованого пароля. Коли користувач створює новий запис для збереження пароля, виконується POST-запит, який передає назву сервісу, логін та згенерований пароль. Сервер перехоплює ці дані, викликає модуль шифрування (AES-256) і лише після цього записує зашифрований рядок у базу даних. Після успішного збереження інтерфейс сторінки оновлюється, і новий запис з'являється у списку збережених паролів користувача.

3. Отримання даних (Відображення паролів). При переході в "Особистий кабінет" виконується GET-запит. Сервер робить вибірку з бази даних виключно тих записів, які належать поточному авторизованому користувачу (фільтрація за user_id), та передає їх у HTML-шаблон для відображення.

4. Адміністратор має можливість керувати системою, для цього буде реалізовано:

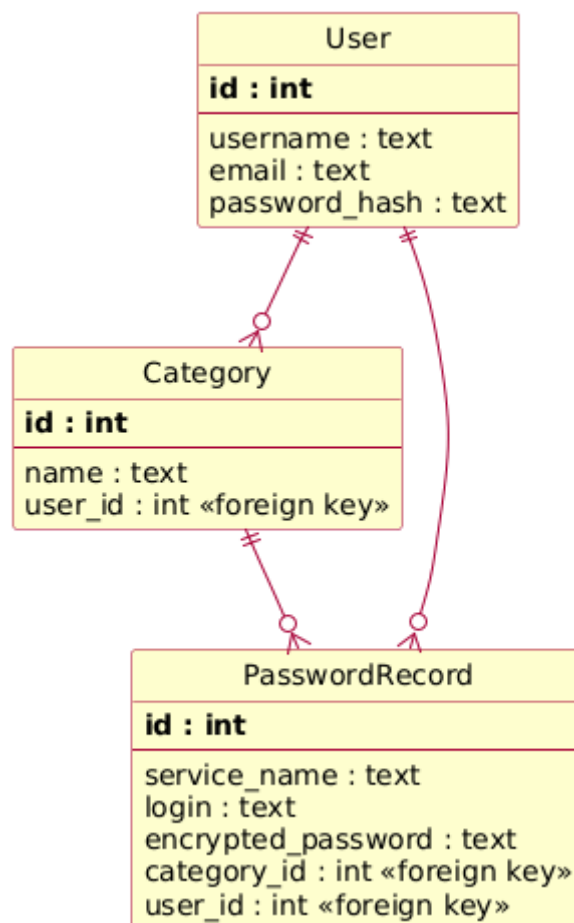
- а. Панель адміністратора для перегляду списку зареєстрованих користувачів.
- б. Можливість блокування або видалення облікових записів.

- с. Доступ до загальної статистики використання системи (без доступу до приватних паролів).

Розділ 6. Розробка бази даних для збереження та отримання інформації

1. Для реалізації системи було обрано реляційну СКБД **PostgreSQL**. Дана система керування базами даних має високий рівень надійності, повністю підтримує принципи ACID, має вбудовані механізми криптографічного захисту та ідеально інтегрується з ORM-системою фреймворку Django. Це критично важливо для додатку, який спеціалізується на збереженні чутливих даних (паролів).

2. Схема бази даних (представлена на Рисунку 9 у Розділі 4) генерується автоматично на основі моделей Django (таблиці User, Category, PasswordRecord).



3. Для оптимізації запитів було створено наступні індекси (генеруються Django ORM автоматично):

а. Таблиця auth_user (Користувачі): індекс за полем id та унікальний індекс за полем username. Використовується для швидкої авторизації.

b. Таблиця `manager_passwordrecord` (Збережені записи): індекс за полем `user_id` (Зовнішній ключ). Використовується для швидкої вибірки та фільтрації всіх збережених паролів, що належать конкретному користувачу під час завантаження його особистого кабінету.

Розділ 7. Інтеграція сторонніх сервісів в систему

Серед сторонніх сервісів в розробленому програмному забезпеченні використовується хмарне сховище для розгортання та хостингу бази даних PostgreSQL (наприклад, сервіс Neon або AWS RDS). Це дозволяє не зберігати базу даних локально, забезпечуючи її доступність та регулярне резервне копіювання.

Комунікація з хмарним сервісом бази даних виконується за допомогою стандартної для Python бібліотеки `psycopg2-binary`. Авторизація та підключення до сервісу відбувається за допомогою Connection String (URI бази даних), який містить унікальний логін, пароль та адресу хоста. Цей рядок підключення зберігається у безпечному файлі конфігурацій (змінні оточення) і передається в налаштування DATABASES фреймворку Django.

Розділ 8. Створення серверної архітектури додатку

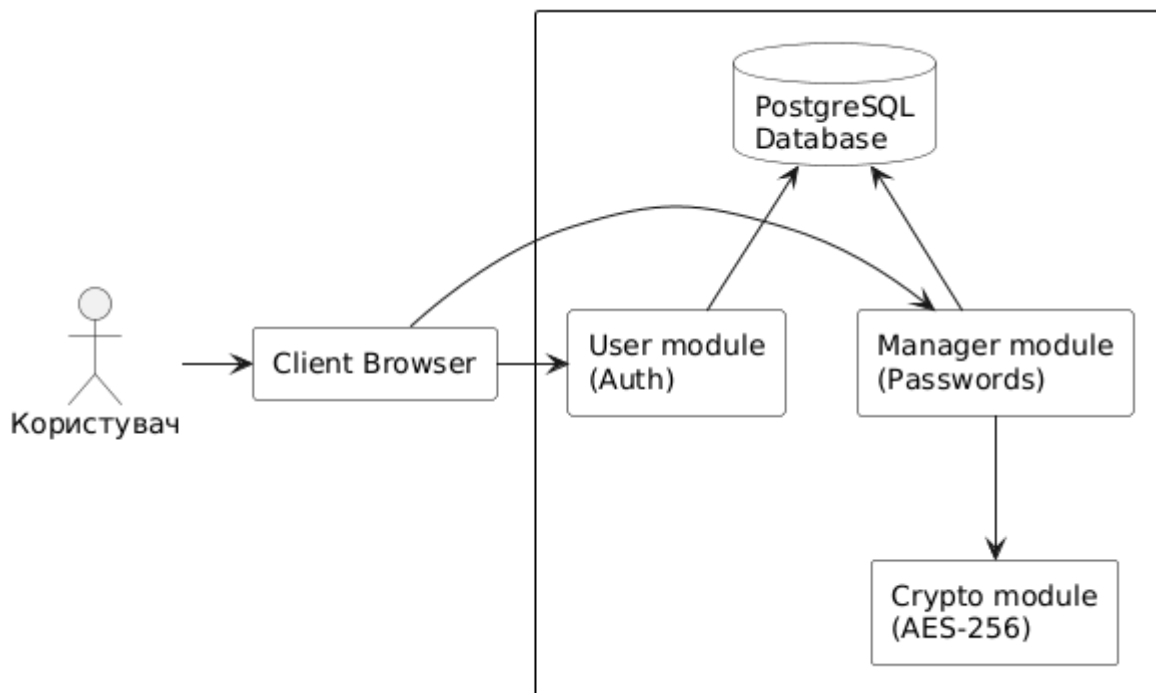
Для розробки серверної частини було використано **монолітну архітектуру** на базі патерну MVT (Model-View-Template), який надає фреймворк Django. Монолітний підхід було обрано, оскільки він дозволяє швидко розгорнути безпечне середовище, спрощує маршрутизацію всередині додатку та забезпечує цілісність даних при роботі з однією базою даних.

В системі присутні наступні основні модулі (класи), які мають чіткий розподіл відповідальності:

1. Модуль аутентифікації. Вбудований модуль Django, який відповідає за роботу з користувачами, хешування їхніх майстер-паролів, керування сесіями та захист маршрутів від неавторизованого доступу.

2. Модуль менеджера. Основний модуль додатку, який відповідає за бізнес-логіку: генерацію безпечних рядків (паролів) за заданими параметрами, відображення інтерфейсу та взаємодію з моделями бази даних.

3. Модуль криптографії. Відповідає за виконання алгоритму AES-256. Взаємодіє з модулем менеджера на етапі запису даних у БД (шифрування) та на етапі їх отримання для відображення (дешифрування).



Розділ 9. Реалізація обробки запитів та взаємодії з базою даних на сервері

У розробленому веб-додатку обробка запитів та взаємодія з базою даних реалізована за допомогою фреймворку Django, який використовує патерн MVT (Model-View-Template).

1. Обробка HTTP-запитів

Усі клієнтські запити перехоплюються модулем маршрутизації (urls.py) та направляються до відповідних функцій-контролерів у модулі views.py.

Система обробляє два основних типи запитів:

- **GET-запити:** Використовуються для отримання сторінок та даних. Наприклад, при переході на сторінку "Особистий кабінет" (/profile/), сервер обробляє GET-запит, звертається до бази даних для підрахунку збережених паролів користувача та повертає згенерований HTML-шаблон.
- **POST-запити:** Використовуються для передачі конфіденційних даних на сервер (авторизація, реєстрація, збереження нового пароля). Всі POST-запити в системі захищені вбудованим механізмом CSRF

(Cross-Site Request Forgery), що унеможливлює підробку запитів сторонніми сайтами.

2. Взаємодія з базою даних через ORM

Для роботи з реляційною базою даних використовується об'єктно-реляційне відображення (Django ORM). Це дозволяє взаємодіяти з даними не через сирі SQL-запити, а через методи класів (моделей) мовою Python, що значно підвищує безпеку (захист від SQL-ін'єкцій).

Приклад взаємодії при збереженні пароля:

1. Сервер отримує POST-запит із форми збереження (назва сервісу, логін, згенерований пароль).
2. Дані проходять базову валідацію.
3. Викликається метод ORM `PasswordRecord.objects.create()`, куди передаються параметри.
4. Перед записом у базу, пароль підлягає шифруванню.
5. Django ORM автоматично генерує безпечний SQL-запит INSERT та додає запис у базу даних із прив'язкою до ідентифікатора поточного авторизованого користувача (`user_id`).

Розділ 10. Тестування та налаштування веб додатку на сервері

1. Налаштування веб-додатку

Налаштування серверної частини зосереджено у головному конфігураційному файлі `settings.py`. Для коректної роботи системи були виконані наступні налаштування:

- **Безпека:** Згенеровано унікальний `SECRET_KEY` для криптографічного підпису сесій та хешування.
- **База даних:** Налаштовано підключення до реляційної бази даних через словник `DATABASES`.
- **Управління доступом:** Для захисту приватних сторінок (кабінет, перевірка, сховище) від неавторизованого доступу застосовано серверний декоратор `@login_required`. Якщо неавторизований користувач намагається зайти на ці сторінки, сервер автоматично перенаправляє його на сторінку входу.

2. Тестування системи

У процесі розробки було проведено комплексне функціональне тестування ключових модулів:

- **Тестування авторизації:** Перевірено механізми реєстрації (з валідацією на співпадіння паролів та унікальність логіна) та входу в систему.
- **Тестування генератора:** Перевірено роботу JavaScript-модуля для генерації паролів. Алгоритм коректно реагує на зміну довжини (від 8 до 32 символів) та включення/виключення певних наборів символів (великі літери, цифри, спецсимволи).
- **Тестування збереження даних:** Перевірено весь цикл передачі даних від клієнта до сервера. Згенеровані паролі успішно передаються через URL-параметри на сторінку збереження, записуються в базу і коректно відображаються в особистому сховищі під відповідним акаунтом.